

2026—2027 学年第一学期

深度学习

Deep Learning

- 第一章 深度学习概述 · 第一章 深度学习概述（课时一 / 共三课时）
- 课程代码 EIT44301C · 大数据2303

授课教师：李家琦

任课教师简介

About the Instructor



李家琦

• 工学博士 • 民盟盟员 • 天津师范大学计算机与信息工程学院 教师

- **教育背景：**哈尔滨工业大学计算机专业博士；新加坡国立大学联合培养博士生
- **研究方向：**大模型知识增强研究及其在智慧教育、AI for Research上的应用
- **工作经历：**曾任科大讯飞与中国科学技术大学联合培养博士后、科大讯飞北京研究院算法专家、科大讯飞高教事业部总工程师
- **学术任职：**中国计算机学会（CCF）自然语言处理专委会执行委员；中国中文信息学会语言与知识计算专委会委员

♦ **工程与科研双背景：**课程中的每个技术点，我都会从“学术原理+工业落地”两个视角来讲。

课程定位

一门“从理论到项目”的深度学习入门课

面向有 Python 与高等数学基础的同学 · 理论 32 学时 + 实践 16 学时 · 结课完成一个可运行的深度学习项目

你准备好了吗？先修能力自查

Prerequisites

必备基础（自查清单）

- Python 语言：函数、列表、字典、循环、类的基本用法
- 高等数学：导数与链式法则、偏导数
- 线性代数：矩阵乘法、向量与维度
- 概率论：概率分布、期望、条件概率

1 Python 生疏 → 开课前自行复习基础语法即可

2 数学遗忘 → 课程讲到对应处会简要回顾

3 没学过机器学习 → 不影响，本课从零讲起

♦ 没有系统学过机器学习也不要紧，这门课从零开始构建知识体系。

课程内容地图：七章一项目

Course Roadmap

第1章 · 3学时 深度学习概述

是什么、怎么来的、搭环境（本课）

第2章 · 3学时 神经网络

感知机、反向传播、PyTorch

第3章 · 9学时 卷积神经网络

卷积、经典架构、图像实战

第4章 · 12学时 循环神经网络

序列建模、LSTM、时序任务

第5章 · 9学时 深度学习优化

优化算法、正则化、调参

第6章 · 6学时 机器视觉应用

Yolo 目标检测、图像生成

第7章 · 6学时 前沿技术

强化学习、扩散模型、大模型

贯穿全程 结课项目

选题、开发到答辩全流程

成绩怎么构成：一半看过程，一半看期末

Assessment

课程目标	课堂表现	随堂练习	实践作业	结课项目	合计
课程目标1：概念与原理	5%	10%	10%	15%	40%
课程目标2：工程与创新	—	—	10%	30%	40%
课程目标3：前沿与迁移	—	—	10%	10%	20%

♦ 过程性评价 50% + 结果性评价 50%；结课项目独占 50% 权重。

教材与参考书

Textbook & References

教材

《深度学习算法与实践》

于子叶主编 · 人民邮电出版社 2020。理论体系完整，课堂主线

参考1

《深度学习》（花书）

Ian Goodfellow 等著 · 2017。深度学习“圣经”，查原理细节

参考2

《PyTorch 深度学习实战》

阿斯顿·张 等著 · 2023。全部上机实践的代码参考

参考3

《机器学习方法》

李航 著 · 清华大学出版社 2025年7月第2版。机器学习方法论权威之作

◆ 实践统一使用 PyTorch——工业界与科研界的主流选择，也是最好上手的框架。

给这门课的四条学习建议

How to Learn

1 代码要亲手敲

看懂 $\neq$ 会写；实践任务独立完成，AI 辅助须注明

2 数学不绕行

多想一步“这个公式在算什么”，本课只要求直观理解

3 每周复盘

把当周代码跑通一次，报错自己先查再问

4 前沿随手记

每周 10 分钟浏览 AI 资讯，项目选题就藏在这里

◆ 深度学习是“练”出来的，不是“听”出来的。

第一章的三个课时怎么上

Today's Plan



- ◆ 课时一结束时，你要能回答：深度学习到底“深”在哪里？

深度学习是什么？

What is Deep Learning?

- 一个问题：机器怎么学会“看”和“听”？
- 定义、与人工智能和机器学习的关系
- 应用版图与能力边界

课堂讨论

2 分钟

先别查资料——你印象中的“人工智能”是什么？

- ? 你在生活里真正用过的 AI 功能有哪些？
- ? 哪一次让你觉得“这机器有点东西”？

提示：刷脸支付、语音助手、修图、短视频推荐、导航、输入法联想.....都算。



你的手机里藏着至少 8 个深度学习模型

DL in Your Pocket

人脸解锁 / 刷脸支付

卷积神经网络逐层提取五官特征

相册搜索 “猫” “海边”

图像分类 + 场景识别

输入法语音转文字

语音识别：序列建模

短视频 “猜你喜欢”

推荐系统：学习兴趣表示

拍照翻译 / 文档扫描

文字检测与识别 OCR

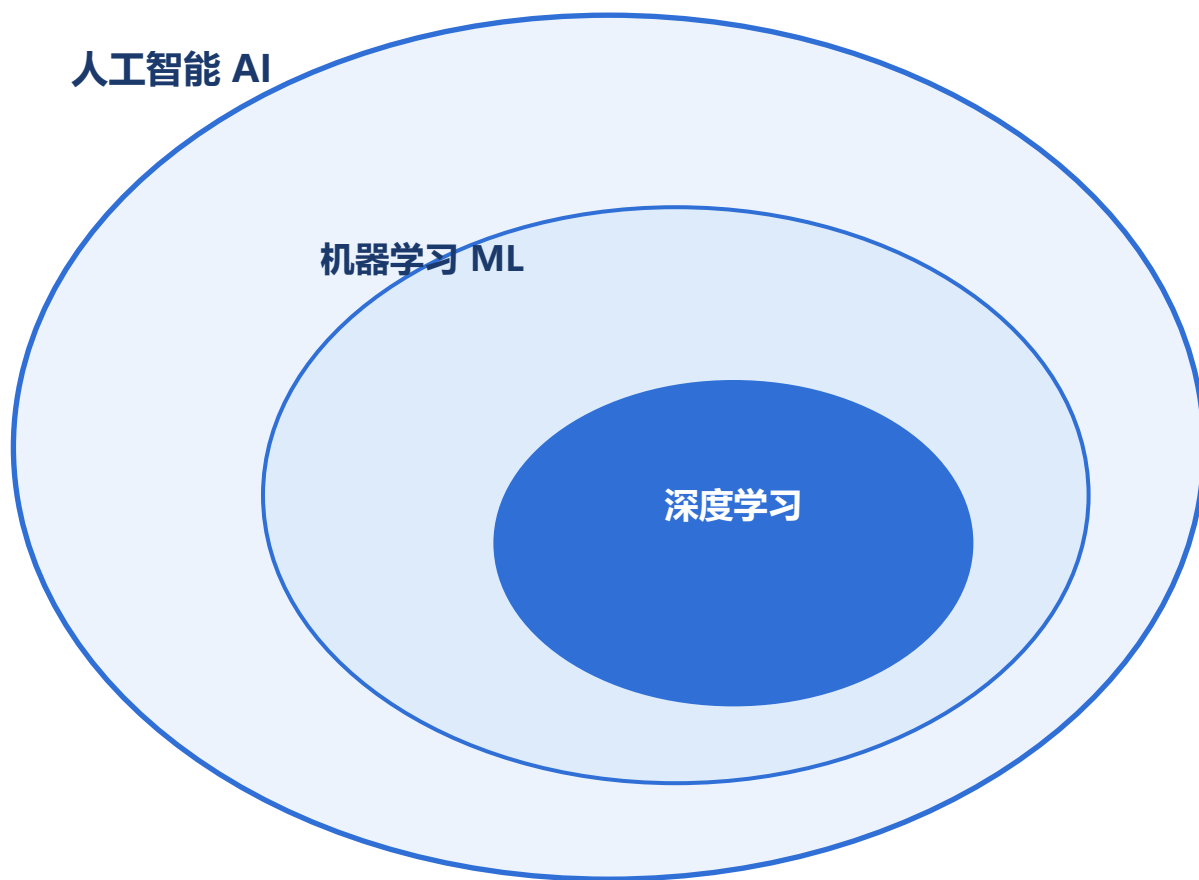
美颜与夜景增强

图像生成与修复网络

◆ 这些功能背后全是深度学习——本质是让机器从数据里 “自己学出” 规则。

AI ⊃ 机器学习 ⊃ 深度学习

The Nesting Relationship



- **人工智能**：让机器表现出智能的一切方法
- **机器学习**：从数据中自动学习规律
- **深度学习**：用多层神经网络自动学习特征表示
- 深度学习是实现机器学习的一种方法，机器学习是实现 AI 的一条路径

◆ 三者是包含关系，不是并列关系——考试常考辨析。

深度学习的定义

通过多层神经网络，自动从数据中学习特征表示的方法

关键词一 “多层”：特征一层层从简单到抽象 · 关键词二 “自动”：不靠人工设计规则 · 关键词三 “表示学习”：学的是数据的内在表达

干货：机器学习的统一框架——三要素

Model · Loss · Optimizer



◆ 记住三要素：任何机器学习方法都能装进"模型-损失-优化"这个框架。

三要素串起一条线：从线性回归到感知机

One Line, Three Boxes

- **模型**： $y = w \cdot x + b$ （一条直线）——最朴素但最基础
- **损失**：预测值和真实值差多远，例如均方误差
- **优化**：沿"下降方向"调整 w 和 b ，让损失变小
- **感知机** = 线性模型 + 阶跃输出 + 误差驱动的更新规则

它们都是同一件事

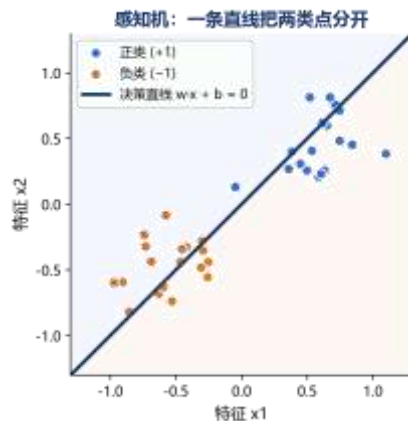
- 线性回归：预测连续数值
- 逻辑回归：预测概率
- 感知机：二分类

◆ 这一页把"线性回归"讲透了，整个机器学习的地基就打下了。

线性模型的"长相"

The Linear Model

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$



w 是权重（每个特征的重要程度）， b 是偏置（直线的偏移）。感知机、线性回归、逻辑回归都是这个式子，区别只在输出层怎么"收尾"。

◆ 这一行公式，是整门课所有模型的"地基"。

深度学习 = 把三要素做到极致

DL as Extreme Version

模型

从直线到多层网络

深度学习的模型 = 千万参数的多层函数，表达能力远超直线

损失

从 MSE 到交叉熵

分类任务的损失函数有讲究（下一页讲）

优化

从解析解到梯度下降

参数太多求不出公式解，只能一步步"下山"逼近

◆ 深度学习没有脱离三要素，只是把每个要素都"卷"到了极致。

感知机的"学习规则"公式

The Perceptron Learning Rule

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (y - \hat{y}) \mathbf{x}$$

y 是标准答案, $\hat{y}$ 是模型预测: 预测对了就不动 ($y - \hat{y} = 0$), 错了就把权重往正确方向推一把, 推的幅度 = 学习率 η × 误差 × 输入。

出处

出处: Rosenblatt, 1958 (同上); 收敛性证明见 Novikoff, 1962.

♦ "错了才学, 错得越多改得越多"——这条公式就是机器学习最初的"学习"。

从一个任务看懂“表示学习”

How a Machine Sees a Cat

- **人类**：一眼认出“这是猫”，却说不出判断步骤
- **传统编程**：写规则“尖耳朵+胡须→猫”？猫一歪头就失效
- **深度学习**：看 10 万张标注图片，自己学出“什么组合是猫”
- **范式转变**：人写规则 → 人给数据+答案，机器自己找规则

机器看到的不是“猫”

- 图像 = 数字矩阵 (如 $224 \times 224 \times 3$)
- 模型输出：每个类别的打分
- 分数最高者即预测结果

◆ **深度学习的核心范式**：数据 + 答案 → 机器自己学出规则。

两种范式：写规则 vs 学规则

Two Paradigms

传统编程

- 输入：数据 + 人写的规则
- 过程：规则作用于数据，产出答案
- 适用：规则清晰可枚举（算税、排序）
- 困境：识别、语音等“只可意会”的任务

VS

机器学习 / 深度学习

- 输入：数据 + 答案（标注样本）
- 过程：算法自动学出规则（模型）
- 适用：规则难以言传的任务（认猫、听音）
- 代价：依赖数据量与算力，可解释性弱

◆ 规则写得出就用传统编程；规则写不出但数据管够，交给深度学习。

“深”到底指什么？

特征一层层加工：像素 → 边缘 → 部件 → 物体

输入 500 万像素，输出一个词“猫”——中间所有特征都是网络自己学出来的；层数越深，能表达的概念越抽象

今天会反复出现的 6 个术语

Key Terms

术语	含义	类比
模型 Model	从数据学出来的“规则体”，可对新输入做预测	见过世面的专家
训练 Training	用数据不断调整模型参数的过程	专家刷题成长
参数 Parameters	模型内部可调节的数值（权重）	脑内神经连接强度
特征 Feature	对判断有用的中间信息	医生看病的检查指标
标注 Label	样本的正确答案	题库的标准答案
推理 Inference	训练好的模型拿去实际使用	专家上岗出诊

♦ 先混个脸熟，第二章起每个词都会展开细讲。

干货：损失函数——给“错得多离谱”打分

Loss Functions

回归任务 → 均方误差 MSE

- 预测房价 3.5 万，实际 3.0 万 → 差 0.5
- $MSE = \text{差值的平方的平均} : (0.5^2)/n$
- 为什么平方？——放大大误差的惩罚，且处处可导

VS

分类任务 → 交叉熵

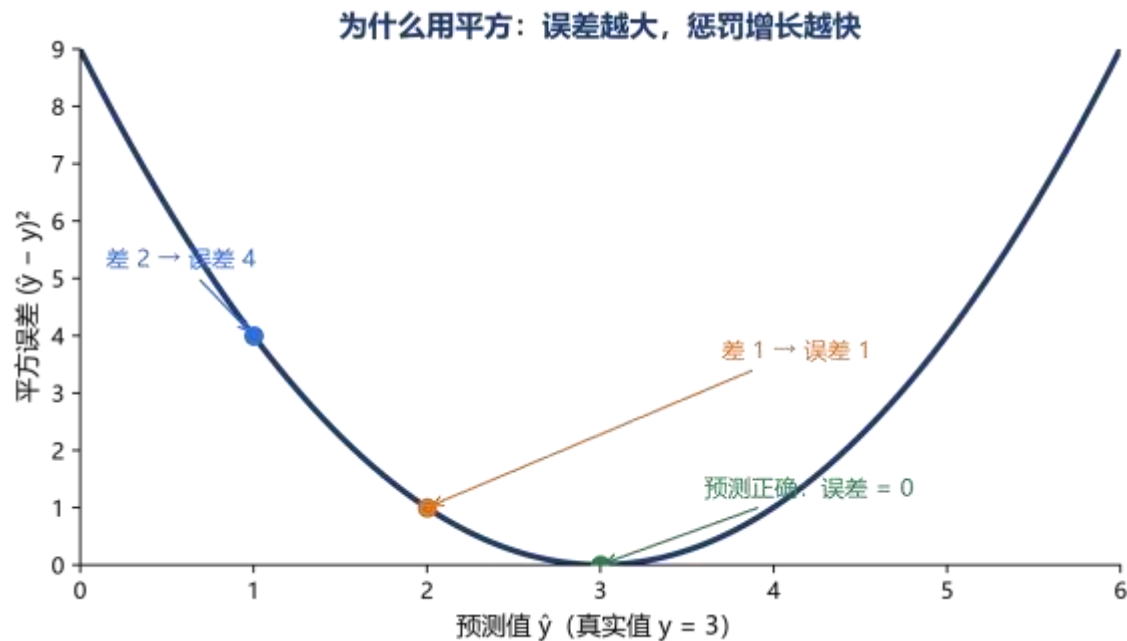
- 预测“猫”的概率 0.9，实际就是猫 → 很满意
- 预测“猫” 0.9，实际不是 → 要狠狠惩罚
- 交叉熵：对“自信地犯错”惩罚特别重

◆ 直觉记忆：回归看“差多少”，分类看“有多自信地错”。

均方误差 MSE 的公式

Mean Squared Error

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



n 个样本，每个样本的预测值与真实值之差求平方，再取平均。平方的两个理由：① 正负误差不会抵消；② 大误差被放大（差 1 的惩罚是差 0.1 的 100 倍）。

◆ MSE = "差值的平方的平均"——公式一分钟就能记住，但"为什么平方"值得想一分钟。

信息熵：不确定性的度量

Shannon Entropy

$$H(X) = - \sum_i p_i \log_2 p_i$$

熵 = 信息量的期望。直觉：抛硬币（两种可能）熵是 1 比特；掷骰子（六种可能）熵约 2.6 比特。

可能情况越多、越不确定，熵越大；确定的事熵为 0。

出处

出处：Shannon, A Mathematical Theory of Communication, Bell System Technical Journal, 1948 —— 信息论的奠基之作。

♦ 理解熵，才能理解交叉熵为什么“惩罚自信的错误”。

交叉熵：分类任务的"成绩单"

Cross-Entropy Loss

$$H(p, q) = - \sum_i p_i \log q_i$$

p 是真实分布（正确答案）， q 是模型预测的概率。当模型对正确答案给出低概率时， $-\log q$ 会非常大——这就是"自信地犯错被狠罚"。

出处

出处：交叉熵源于信息论（Shannon 1948）；作为分类损失的标准做法见 Goodfellow et al., Deep Learning, 2016 (花书 §6.2)。

♦ 交叉熵 $\geq$ 熵，且只在预测完全正确时相等——这是它的数学本质。

一句话记住两个损失

Two Losses in One Line

- **MSE**: 差 0.1 和差 1.0, 惩罚差了 100 倍 (平方放大)
- **交叉熵**: "很有把握却错了"是最不能容忍的
- **共同点**: 损失永远 ≥ 0 ; 损失 = 0 意味着完美预测
- **损失函数是模型的"成绩单"**——训练的一切都是为了把这张成绩单的分数降下去

◆ 接下来要回答: 成绩单有了, 怎么把分数降下去? ——梯度下降, 第二章正式讲。

干货：优化 = 下山（直觉版）

损失是山坡，参数是你的位置，梯度告诉你往哪走

山越高=损失越大 · 沿"下坡方向"挪一小步，损失就小一点 · 反复走，直到谷底——这就是梯度下降的直觉，公式第二章再给

梯度下降的更新公式

Gradient Descent Update

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

θ 是全部参数， η 是学习率， $\nabla_{\theta} L$ 是损失对参数的梯度（山坡最陡的方向）。减号 = 朝下坡走。这一行，就是深度学习训练循环里 `opt.step()` 背后的数学。

出处

出处：梯度下降思想可追溯至 Cauchy 1847；神经网络训练中的系统使用见 Rumelhart, Hinton & Williams, Nature, 1986.

◆ 第三章你会亲眼看到：PyTorch 里的一行 `opt.step()`，背后就是这条公式。

互动：看小球自己"下山"

Lab: Gradient Descent Ball



课堂怎么玩

- 1 默认 $\eta=0.1$ 自动下山，看小球滚向谷底
- 2 把学习率拖到 0.005——"米粒步"半天走不到
- 3 拖到 1.2——小球在谷底来回蹦甚至飞出屏幕

看什么：同一条下山路，步幅太小磨蹭、太大翻车——学习率是深度学习第一个超参数。

◆ 公式留给第二章，但"学习率太大为什么会震荡"这个直觉，今天先种下。

深度学习的应用版图

Application Landscape

计算机视觉

图像分类 · 目标检测 · 图像生成 (第3、6章实战)

自然语言处理

机器翻译 · 文本生成 · 大语言模型 (第4、7章)

语音处理

语音转文字 · 语音合成 · 语音唤醒

推荐系统

个性化推荐 · 广告投放 · 内容分发

科学与医疗

医学影像 · 蛋白质结构预测 · 药物发现

自动驾驶与机器人

感知 · 决策 · 控制全链路

◆ 别被领域多吓到：底层模型就那几类，学通了到处通用。

案例：深度学习如何辅助医生看 CT

Case Study

传统方式 vs 深度学习辅助

- 人工阅片：医生逐层查看数百张 CT 切片
- 疲劳与经验差异带来漏诊风险
- 深度学习：扫描全片，自动标注可疑结节
- 医生复核 AI 标记，只看重点区域

1 第3章 CNN：正是影像识别的核心模型

2 跨学科协作：计算机 × 医学，各司其职

3 定位：AI 是辅助，诊断责任仍在医生

◆ 技术价值在于“把人从重复劳动中解放出来”，而不是替代人。

冷静看待：深度学习不是万能的

What DL Cannot Do (Yet)

擅长做的事

- 有海量标注数据的感知类任务
- 规则难写的模式识别：图像、语音、文本
- 允许一定错误率、有人兜底的场景

VS

不擅长的事

- 数据稀缺且昂贵的任务（标注贵）
- 要求严格可解释的场景（司法、医疗决策）
- 严格逻辑推理与数学证明（仍在突破）

◆ 判断任务适不适合深度学习，是本课要培养的核心判断力之一。

干货：过拟合与欠拟合——“背答案” vs “没学会”

Overfitting & Underfitting

- **欠拟合**：模型太简单，训练集都没学好（考试范围都没覆盖）
- **过拟合**：模型太复杂，把训练集的“噪声”也背下来了（只会背原题）
- **健康状态**：训练误差低，新数据（测试集）误差也低 = 泛化好
- **机器学习的终极目标不是背题，而是“举一反三”——这叫泛化**

背答案的学生

- 训练题全对
- 换新题就懵
- 本质：记住了噪声

◆ 记住三词：欠拟合（太简单）、过拟合（背答案）、泛化（举一反三）。

数据为什么必须分三份？

Train / Validation / Test

数据集	用途	比喻
训练集 Train	用来调整模型参数（学习）	学生平时刷的习题册
验证集 Validation	用来挑超参数、选模型（调优）	模拟考（可反复用）
测试集 Test	最终验收，只能看一次	期末考试（不能偷看）

- ♦ 测试集是“期末考试”：提前看过答案，成绩就没有意义了。

互动：亲手"造"一次过拟合

Lab: Overfitting Playground



课堂怎么玩

- 1 拖"多项式阶数"从 1 到 15，看拟合曲线从直尺变成疯狂扭动
- 2 盯住右侧两条误差曲线：训练一直降、测试先降后升
- 3 把噪声调大，观察"分叉点"是不是来得更早

看什么：阶数低=欠拟合，阶数高=过拟合，中间那一段=泛化最好——自己动手找到"甜区"。

◆ 刚才的曲线不是示意图，是真实数据、真实拟合——过拟合不是抽象名词。

泛化性

好模型 = 训练误差低 + 测试误差也低

预防过拟合的常用武器（正则化、Dropout、早停）将在第五章正式登场——今天先建立"泛化"这个北极星

随堂练习

4 分钟

以下任务，该用深度学习还是传统编程？说明理由

- ? 判断一个身份证号是否合法（校验位规则）
- ? 从街景照片中找出所有外卖电动车
- ? 计算房贷月供（等额本息公式）
- ? 判断 X 光片里是否有骨折征兆

提示：规则能枚举 → 传统编程；规则难写、模式感知类 → 深度学习。

小结：五个带走的知识点

Key Takeaways



定义

多层神经网络自动学习特征表示的方法



关系

人工智能 $\supset$ 机器学习 $\supset$ 深度学习



范式

数据 + 答案 $\rightarrow$ 机器自己学出规则



应用与边界

感知类任务强项；可解释与逻辑推理仍是短板



判断力

规则写得出一用传统编程；数据多、规则难写一用深度学习

◆ 稍作休息后，我们回看这条技术路线走过的六十年。

课后小任务

After Class

1 阅读教材第一章

于子叶《深度学习算法与实践》第 3 章，对照今天内容

2 记录 3 个 AI 应用

写下本周遇到的 3 个深度学习应用，猜测所属领域

3 预告思考题

“为什么数据越多，深度学习越强，传统方法却会饱和？”

♦ 思考题不需要交，但课时二开场会抽人回答。

课时一·到此结束

休息 10分钟

课时二 · 开场之问

机器“看懂”世界的能力，是被人教會的，还是自己学会的？

——这是理解深度学习与传统机器学习分水岭的钥匙

三次浪潮与两次寒冬

A History of Three Waves

- 1958 感知机 → 1986 反向传播 → 2012 AlexNet → 2017 Transformer
- 为什么深度学习“死”了两次又活了两次？
- 今天的大模型从哪里来

1958 感知机

Rosenblatt · 感知机

开局即巅峰：感知机问世

- 罗森布拉特提出感知机（Perceptron）：第一个可以“学习”的数学模型
- 硬件 Mark I：400 个光电管 + 电位器，真能学会分辨简单图形
- 《纽约时报》报道：“注定能走路、说话、看见、书写、自我复制并意识到自己的存在”

“感知机是第一条“从数据中学习”的技术路线——整门课都建立在它身上。”

出处： Rosenblatt, The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain, Psychological Review, 1958

1969 XOR问题

Minsky & Papert

第一次寒冬：一本书浇灭一个领域

- 《感知机》一书证明：单层感知机连“异或”这种简单问题都解决不了
- 多层网络潜力虽在，但当时无人能训练：没算法、没数据、没算力
- AI 经费大规模转向符号主义；神经网络研究停滞近 15 年

“寒冬的根源不是想法错了，而是“当时的工具配不上当时的想法”。”

出处：Minsky & Papert, Perceptrons: An Introduction to Computational Geometry, MIT Press, 1969

插曲：为什么一个“异或”能杀死一个领域

Why XOR Matters

异或问题

- 输入两个开关状态 (0/1)，输出：相同为 0，不同为 1
- 画在坐标纸上：四个点，无法用一条直线分开
- 单层感知机 = 只能画一条直线（线性可分）
- 多层网络 = 可以画曲线、画多条线（非线性）

1 把问题画出来：○○在左下、右上，●●在另两角

2 单层：一条直线，怎么摆都切不开

3 两层：先横切一刀再竖切一刀，轻松解决

◆ “线性不可分”是第一次寒冬的技术死穴；多隐层是解药，但要会训练才行。

1986 BP算法

Rumelhart • Hinton •
Williams

复兴之火：反向传播算法

- **反向传播** (Backpropagation)：让多层网络每层都能“分摊错误、各自修正”
- 多层前馈网络从此可以高效训练，神经网络的“死穴”被解开
- **今天训练 PyTorch 模型，底层仍然是它**——你第三课时装的环境就是来跑它的

“反向传播是全课程的算法基石，第二章将用整整一节把它讲透。”

出处： Rumelhart, Hinton & Williams, Learning Representations by Back-Propagating Errors, Nature 323, 1986

为什么 90 年代又 “凉” 了？

The Second Winter

- **数据太少**：互联网刚起步，百万级标注数据想都不敢想
- **算力太弱**：CPU 训练一个中等网络要几天到几周
- **方法之争**：SVM 等浅层方法在小数据上又快又好，风头更盛
- **深层网络在当时 “训不动、训不好”，领域再次降温**

◆ “巧妇难为无米之炊”：深度学习天生需要大数据与大算力。

2006

Hinton · 深度信念网络

命名时刻：“深度学习”登场

- Hinton 提出深度信念网络（DBN）与逐层预训练，深网络终于“训得动”
- “Deep Learning” 一词正式流行，领域有了自己的名字
- 学界重新捡起神经网络：这一次，名字、方法、信心都齐了

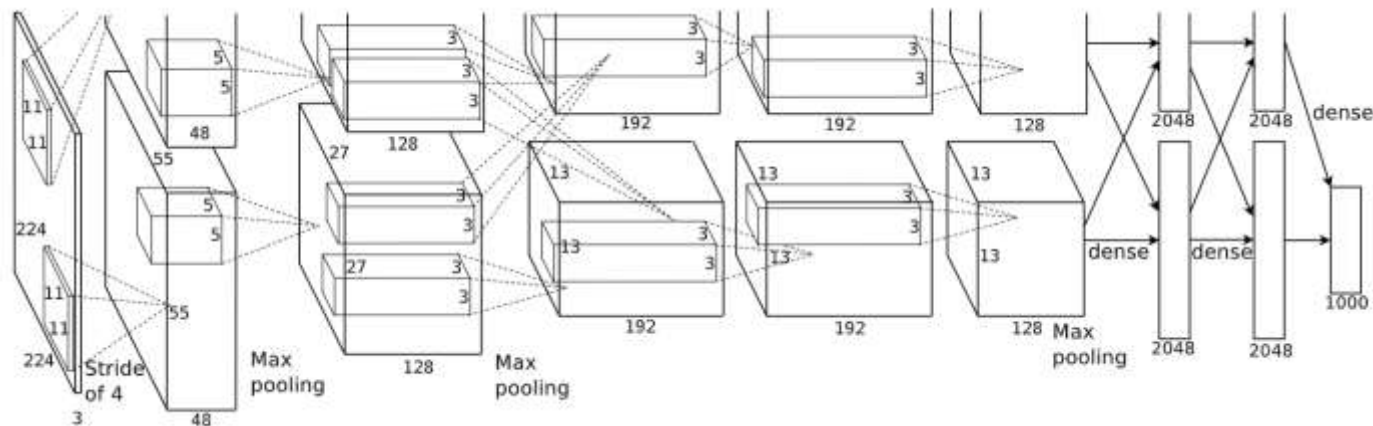
“2006 是概念转折点，真正的爆发还差一场大赛——以及足够的燃料。”

出处：Hinton, Osindero & Teh, A Fast Learning Algorithm for Deep Belief Nets, Neural Computation, 2006

2012年 AlexNet

Krizhevsky · Sutskever · Hinton

- “我们训练了一个大型深度卷积神经网络，去区分 ImageNet 的 120 万张高清图片”
- 论文通篇没有“人工设计特征”的章节——这在当时是离经叛道的写法
- 以往历届冠军方案的核心章节，恰恰是精心设计的特征工程



“不设计特征，让网络自己学特征——AlexNet 用结果证明这条路走得通。”

出处: Krizhevsky, Sutskever & Hinton, ImageNet Classification with Deep Convolutional Neural Networks, NeurIPS, 2012

2012 ImageNet: 一夜改写历史

The AlexNet Moment

120万

标注图片

ImageNet 数据集规模

15.3%

AlexNet 错误率

第二名低约 10 个百分点

2×GTX580

训练硬件

两块游戏显卡训练一周

- ◆ 深度学习自此成为计算机视觉的默认选择，并迅速席卷其他方向。

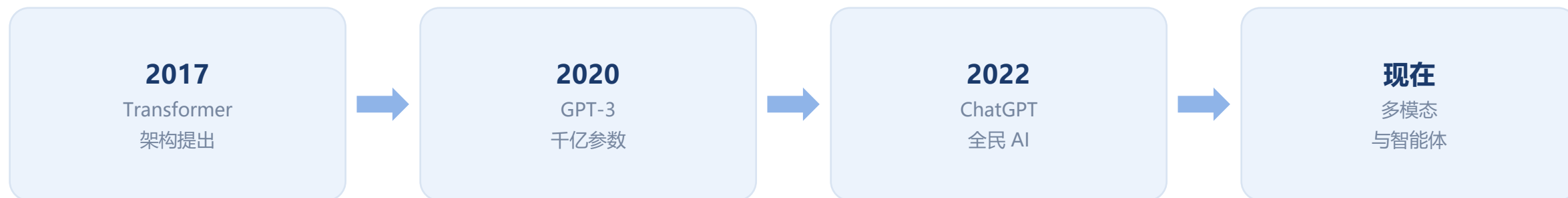
出处: Krizhevsky, Sutskever & Hinton, ImageNet Classification with Deep Convolutional Neural Networks, NeurIPS, 2012

从 Transformer 到大模型时代

2017 → Now

回望 2012 点燃之后的那五年：视觉、语言、生成、博弈四条线同时加速

2013 词向量让语言 “可计算” · 2014 GAN 让机器会 “创作” · 2015 ResNet 152 层超越人类 · 2016 AlphaGo 战胜人类冠军



- ♦ **大模型的底层仍是深度学习：你正在学的内容，就是这波浪潮的技术地基。**（出处：Vaswani et al., Attention Is All You Need, NeurIPS, 2017）

一图串起 60 余年

Timeline

1958

感知机
Rosenblatt

1986

反向传播
Hinton 等

2012

AlexNet 夺冠
ImageNet

1969

第一次寒冬
《感知机》一书

2006

“深度学习”命名
深度信念网络

2017

Transformer
Attention 机制

◆ 记年表不如记逻辑：算法先行 → 数据算力跟上 → 应用爆发。

课堂讨论

3 分钟

深度学习“死”了两次，又活了两次，靠的是什么？

- ? 如果你是 1995 年的研究者，会继续做神经网络还是转行？
- ? 三次浪潮的引爆点分别对应“想法、算法、数据算力”中的哪个？

提示：1958 靠想法，1986 靠算法，2012 靠数据与算力——三要素凑齐才有今天。



爆发三要素：数据 · 算力 · 算法

Data, Compute, Algorithms

要素一

数据

互联网与传感器带来海量标注数据；数据越多
，深度学习优势越大

要素二

算力

GPU 并行计算把训练从“几周”压到“几小时”
；我国智算规模居世界前列

要素三

算法

ReLU、Dropout、BatchNorm、残差连接等
，让深网络稳定可训

◆ 三者缺一不可：这是判断“下一个风口”的通用框架。

时代坐标：中国的 AI 突破

DL in China

- **大模型百花齐放：**DeepSeek、GLM、Kimi、Qwen等进入世界第一梯队
- **开源贡献显著：**国产开源模型被全球开发者使用与二次开发
- **国家战略：**深度学习是新一代人工智能发展规划的核心方向
- **把个人所学融入国家科技自立自强，是这一代计算机人的机遇**

从跟跑到并跑、领跑

- 计算机视觉、语音识别全球领先
- 开源社区与国际同台竞技
- 算力基建规模居世界前列

◆ 你们现在学的每一个模型，都连接着国家的 AI 战略版图。

特征学习革命

The Representation Learning Revolution

- 什么是特征工程？它为什么昂贵？
- 端到端学习：联合优化的威力
- 预训练 + 微调：深度学习的“普惠红利”

什么是特征工程？为什么要“消灭”它

What is Feature Engineering

- **特征**：对判断有用的中间信息，是“把原始数据翻译给算法听”
- **特征工程**：由人类专家手工设计这些翻译规则
- **瓶颈**：换任务就要重新设计；上限取决于人的想象力
- “谁懂领域知识，谁设计特征”——深度学习要打破的就是这句话

传统视觉特征举例

- SIFT：角点与局部特征
- HOG：物体轮廓梯度
- 颜色直方图、纹理特征

◆ 特征工程是把专家经验“翻译”给算法；深度学习让机器自学翻译。

一个类比：老中医与 AI 诊脉

An Analogy

- **传统机器学习像“师带徒”**：师傅把“看舌苔、数脉象”的口诀一条条教给徒弟（人工特征）
- **深度学习像“见万病人”**：徒弟看够一万份病例，自己悟出规律（自动特征）
- **口诀的天花板**：师傅没总结过的症状，徒弟就认不出来
- **数据的力量**：病例够多时，“悟出来”的规律往往超越口诀

◆ **口诀 = 人工特征工程；海量见诊 = 表示学习。悟性来自数据。**

传统机器学习流水线：六道工序

Traditional Pipeline



- ◆ 每个环节独立优化，误差层层累积；任何一环做不好，全盘皆输。

端到端学习：一条流水线变一块网络

End-to-End Learning

传统流程

- 多个环节各自为政、独立优化
- 误差在环节间层层累积
- 换任务 = 重新设计整条流水线
- 人对“中间环节”负责

VS

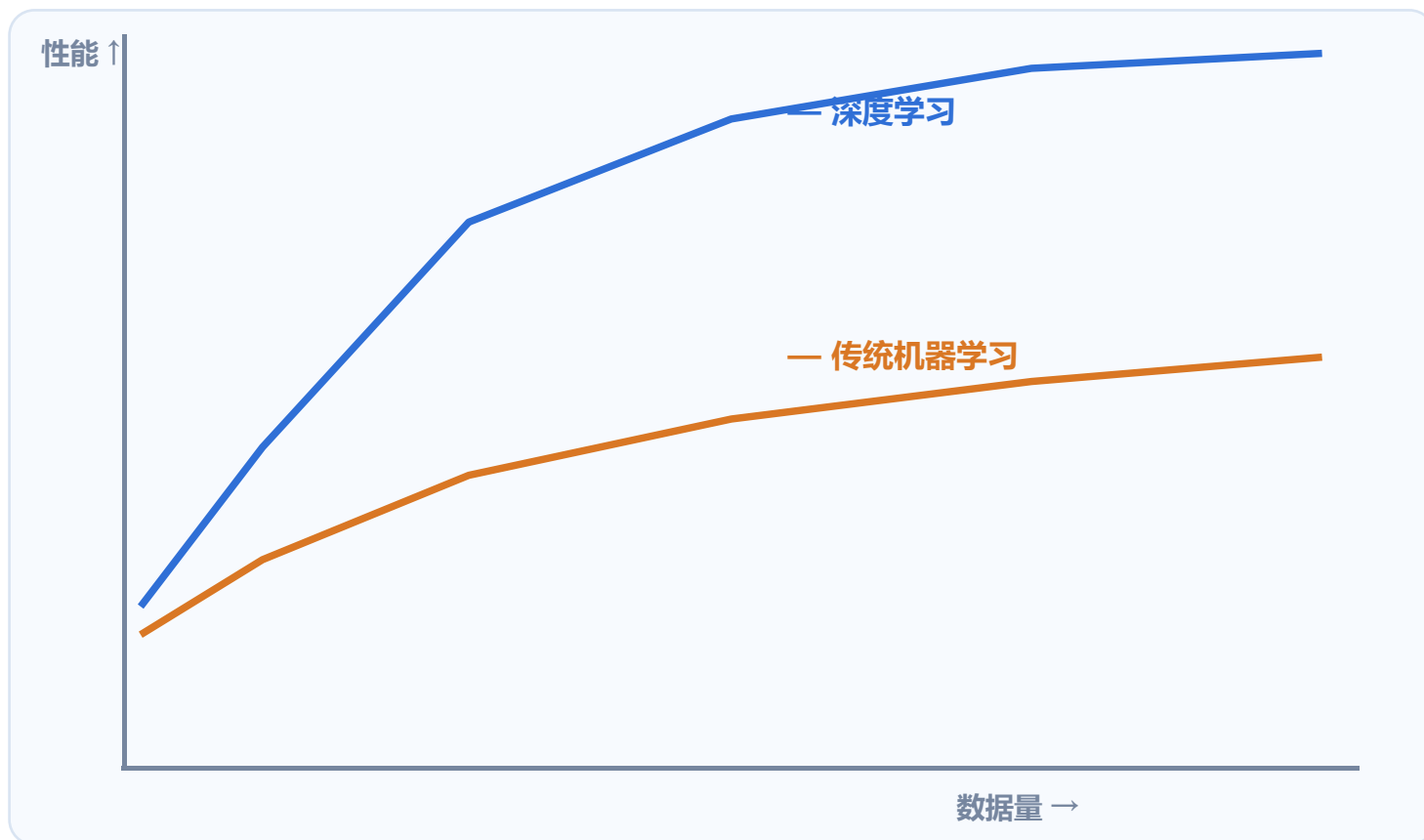
端到端深度学习

- 原始数据进，最终结果出
- 整体联合优化，误差统一回传修正
- 换任务 = 换数据重新训练
- 模型对“最终结果”负责

◆ “端到端”的优势是联合优化：所有参数朝同一个目标一起调整。

数据规模决定胜负：那道著名的交叉曲线

Scaling with Data



- **传统机器学习**：受人工特征上限约束，数据再多也趋于饱和
- **深度学习**：表示容量随网络规模增长，数据越大性能越强
- 两条曲线的交叉点，就是 2012 年后业界全面转向的经济学解释

◆ 思考题答案：不是深度学习突然变聪明，是“燃料”充足后它的上限更高。

三个概念各就各位

表示学习是目的 · 神经网络是手段 · “深度” 是程度

表示学习：自动学出对任务有用的特征表示；

神经网络：实现表示学习的函数结构；

深度：层数多、特征加工链路长

意外红利：预训练 + 微调范式

Pre-training & Fine-tuning



- ◆ “站在大模型的肩膀上做事” ——这个范式让深度学习飞入寻常百姓家。

代价清单：黑盒带来的新问题

The Price of Representation Learning

我们得到了

- 免人工特征，迁移能力强（预训练+微调）
- 性能上限随数据与模型规模持续提升
- 视觉、语音、语言用同一套范式

VS

我们也付出了

- 可解释性差：很难说清网络为何这样判断
- 依赖数据与算力：小团队负担重
- 鲁棒性问题：对抗样本一改就错

◆ 第五章“深度学习优化”会正面处理这些代价，第七章前沿也会回响。

案例：一张“骗过 AI”的熊猫

Adversarial Examples

对抗样本攻击

- 给熊猫照片加一层人眼看不见的噪声
- 模型却以 99.3% 置信度输出 “长臂猿”
- 原因：模型学到的特征与人类直觉不同
- 自动驾驶、人脸门禁都面临同类风险

1 理解“黑盒”：模型内部特征不等于人类概念

2 认识风险：安全攸关场景需人工兜底

3 了解对策：对抗训练、输入检测（第5章延伸）

◆ “模型会看”不等于“模型像人一样看”——这是理解 AI 局限的关键一课。

课堂讨论

微辩论：特征工程师会失业吗？

- ? 正方：端到端学习终将取代一切人工特征设计
- ? 反方：领域知识永远有不可替代的价值

提示：参考视角：医学 AI 仍依赖医生的先验知识来定义任务与检查数据质量。



下一课时预告：亲手点亮你的第一块 GPU

Coming Up

Step 1

装 Anaconda

Python 环境管理工具，一次安装、长期受益

Step 2

建 PyTorch 环境

创建专属虚拟环境，安装 PyTorch

Step 3

跑第一个程序

张量创建、GPU 检测、Hello Deep Learning

◆ 理论收官，实践开场——课时三见。

课时二小结：一场范式革命

Key Takeaways



三次浪潮

1958 想法、1986 算法、2012 数据算力，三要素齐备才有今天



特征工程

人写特征，任务一换就重来，上限被人锁死



端到端

联合优化取代六道工序的误差累积



预训练+微调

大数据学通用表示，小数据迁移应用



代价

黑盒、吃算力、对抗样本——第五章起正面应对

◆ 理论框架已立起。课时三，把 PyTorch 装进你的电脑。

课时二 · 到此结束

休息 5 分钟 · 课时三将进入上机实操

上机实践：搭建 PyTorch 环境

Lab: Environment Setup

- Anaconda 安装与环境创建
- 验证：张量创建、GPU 检测
- 第一个深度学习程序：Hello Deep Learning

本课时的三个实践目标

Lab Objectives

1 环境就绪

Anaconda + 虚拟环境 dl2026 + PyTorch 全部安装完成

2 验证通过

张量创建、基本运算、GPU 检测三个验证程序运行成功

3 记录沉淀

安装步骤与报错解决过程写入实验记录，作为第一次实践作业

♦ 今天下课时，你的电脑就是一台“深度学习就绪”的机器。

为什么装个环境这么讲究？

Why Environment Management

- **痛点一：** 版本地狱——课程 A 要 Python 3.10，课程 B 要 3.8，互相打架
- **痛点二：** 依赖污染——装了某个包，另一个项目突然跑不起来了
- **痛点三：** 换台电脑全部重来，环境无法“搬家”
- **解法：** Anaconda 统一管理 Python 本体 + 虚拟环境彼此隔离

◆ 一次把环境管好，整个学期省心——工程思维从第一课开始。

Anaconda: 三合一的 Python 工作台

What is Anaconda

组件一

Python 本体

解释器与标准库，版本由 conda 统一管理

组件二

conda 包管理

一条命令安装/卸载/更新包，自动处理依赖

组件三

环境管理

为每个项目创建独立隔离的 Python 环境

◆ 装 Anaconda ≠ 装 Python: 它管的是“你的所有 Python”。

Step 1 · 安装 Anaconda (约 5 分钟)

Install Anaconda

1 下载

官网 anaconda.com 或国内清华镜像, 选 Windows 64-bit 图形版

2 安装

一路默认; 仅勾选 “Add to PATH” 可选, 其余不动

3 验证

开始菜单打开 “Anaconda Prompt”, 输入 `conda --version`

♦ 下载慢就用清华镜像: mirrors.tuna.tsinghua.edu.cn。

验证安装：第一条命令

Verify Installation

Anaconda Prompt

打开 Anaconda Prompt, 输入:

```
conda --version
```

期望输出 (版本号可能不同) :

```
conda 26.x.x
```

- ◆ 能看到版本号 = 安装成功; 报 “不是内部或外部命令” = PATH 问题, 看第 89 页。

配置清华镜像：下载提速 10 倍

Mirror Configuration

可选 · 网速慢再做

在 Anaconda Prompt 中依次执行：

```
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/main/  
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/free/  
conda config --set show_channel_urls yes
```

◆ 校园网环境一般不慢，可跳过；后续下载卡住再回来配。

conda 高频命令速查（收藏用）

conda Cheat Sheet

命令	作用
<code>conda create -n 名字 python=3.10</code>	创建新虚拟环境
<code>conda activate 名字</code>	进入（激活）环境
<code>conda deactivate</code>	退出当前环境
<code>conda env list</code>	列出所有环境
<code>conda install 包名 / pip install 包名</code>	在当前环境安装包
<code>conda remove -n 名字 --all</code>	删除整个环境

♦ 先混脸熟：今天只用前三条，其余用到再查。

Step 2 · 创建课程专属环境

Create Environment

Anaconda Prompt

```
conda create -n dl2026 python=3.10  
# dl2026 = Deep Learning 2025, 环境名自取, 建议好记  
# 过程询问 Proceed [y/n]? 时输入 y 回车
```

- ◆ 一个课程一个环境：以后装什么都不会污染别的课程。

虚拟环境：为什么每个项目一个“房间”

Why Virtual Environments

- **隔离**：每个环境的包互不可见，版本冲突成为历史
- **可复现**：“在我电脑上能跑”不再是玄学，环境可以导出
- **可丢弃**：环境坏了删掉重建，3 分钟恢复，不用重装系统
- **工程习惯**：进项目先建环境，如同开工先搭脚手架

dl2026 环境

- Python 3.10 专属
- PyTorch 装在这里
- 删掉重来零负担

◆ `conda activate dl2026` —— 之后每节课的第一次操作。

激活环境与检查 Python

Activate & Check

Anaconda Prompt

```
conda activate dl2026
```

```
python --version
```

成功标志：命令行开头出现 (dl2026)

(dl2026) C:\Users\你的用户名>

- ◆ 看行首的 (dl2026)：在环境里，怎么装都不怕。

随堂练习

3 分钟

进度检查：完成“创建环境”的同学举手

- ? 已完成：帮助左右邻座检查行首是否显示 (dl2026)
- ? 未完成：常见原因——用了普通 CMD 而不是 Anaconda Prompt

提示：必须用 Anaconda Prompt（开始菜单搜 anaconda），普通 CMD 找不到 conda 命令。

认识 PyTorch：你将天天打交道的三件套

PyTorch Core

组件一

Tensor 张量

多维数组，数据的载体；会自动跑在 CPU 或 GPU 上

组件二

Autograd

自动求导：反向传播不用手写，一行
`backward()`

组件三

nn 模块

现成的网络层与模型组件，像搭积木一样搭网络

◆ 张量是数据、Autograd 是求导、nn 是积木——第二章起逐个登场。

为什么本课选 PyTorch?

PyTorch vs TensorFlow

PyTorch

- 代码风格贴近 Python，上手最快
- 科研界主流：顶会论文代码九成以上
- 工业界份额逐年上升，就业加分
- 本课全部实践基于 PyTorch

VS

TensorFlow

- 部署生态成熟（手机、嵌入式）
- Keras 接口对新手友好
- 国内部分企业存量项目仍在用
- 学会 PyTorch 后迁移半天可上手

◆ 先精一门：PyTorch 是当前“学习成本最低、生态最活”的选择。

安装命令哪里来？官网安装向导

PyTorch Build

Stable (2.13.0)

Preview (Nightly)

Your OS

Linux

Mac

Windows

Package

Pip

LibTorch

Source

Language

Python

C++ / Java

Compute Platform

CUDA 12.6

CUDA 13.0

CUDA 13.2

~~ROCm 7.2~~

CPU

Run this Command:

```
pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu126
```

Step 3 · 安装 PyTorch

Install PyTorch

Anaconda Prompt

先激活环境，再安装（CPU 版，人人能装）：

```
conda activate dl2026
```

```
conda install pytorch torchvision torchaudio cpuonly -c pytorch
```

有 NVIDIA 显卡的同学，用官网生成的 CUDA 版命令替换上面第三行

♦ CPU 版完全够本学期前半程使用；CUDA 版下载大，量力而行。

验证安装: import torch

Verify PyTorch

Python 交互式解释器

```
python
>>> import torch
>>> torch.__version__
'2.x.x'
>>> torch.cuda.is_available()
False # CPU 版返回 False 是正常的
```

◆ **import torch 不报错 = 安装成功**; is_available() 我们下一页专门解释。

Jupyter Notebook: 深度学习的“草稿纸”

Meet Jupyter

- **是什么：** 浏览器里的交互式编程环境，代码 + 运行结果 + 笔记混排
- **为什么用：** 边写边看结果，数据处理、画图、调试都直观
- **行业地位：** 数据分析与深度学习实验的事实标准工具
- **本课所有上机实践的代码，默认都在 Notebook 里完成**

◆ **Notebook = 单元格逐段执行；实验报告直接导出，一物两用。**

Jupyter 生存必备 5 招

Jupyter Basics

1 启动

激活环境后输入 jupyter notebook，浏览器自动打开

2 两种模式

绿框编辑模式写代码；蓝框命令模式按快捷键

3 运行

Shift + Enter 运行当前格并跳到下一格

4 补一格

命令模式按 A（上方）或 B（下方）插入单元格

5 救命键

命令模式按 H 查看全部快捷键；Kernel 菜单可重启

♦ 记住 Shift+Enter 就能开工，其余用到再查。

CPU 与 GPU：为什么训练要用显卡

CPU vs GPU

CPU 中央处理器

- 核心少（几核到几十核），单核强
- 擅长复杂逻辑、串行任务
- 类比：几位教授，解题又快又深

VS

GPU 图形处理器

- 核心数千个，单核简单
- 擅长海量重复的矩阵运算
- 类比：上千名小学生，同时做加法飞快

◆ 深度学习 ≈ 海量矩阵运算，正好是 GPU 的主场——这解释了 2012 年革命。

检测你的深度学习“燃料”

GPU Check

Python 交互式解释器

```
>>> import torch
```

```
>>> torch.cuda.is_available()
```

```
True # 有 NVIDIA 显卡 + CUDA 版 PyTorch
```

```
False # CPU 版或无 NVIDIA 显卡 (本学期前半程够用)
```

```
>>> torch.cuda.get_device_name(0) # True 时查看显卡型号
```

- ◆ 返回 False 不影响本课：第 5 章前的所有实验 CPU 都跑得动。

没有独显怎么办？三条免费路线

Free GPU Options

路线一

Google Colab

浏览器里的免费 GPU Notebook

路线二

Kaggle Notebook

每周 30 小时免费 GPU，注册即用

路线三

百度飞桨 AI Studio

国内直连，每天 8 点免费算力卡

◆ 硬件不是门槛：手段总比困难多，够用就行。

张量 Tensor: 深度学习的“通用容器”

标量 0维 → 向量 1维 → 矩阵 2维 → 张量 N维

一张灰度图 = 2维张量 · 一张彩色图 = 3维 (高×宽×通道) · 一批 32 张彩色图 = 4维——第3章你会天天见它

动手：创建你的第一个张量

First Tensor

Notebook 单元格

```
>>> import torch
>>> x = torch.tensor([[1, 2, 3], [4, 5, 6]])
>>> x
tensor([[1, 2, 3],
        [4, 5, 6]])
>>> x.shape
torch.Size([2, 3])    # 2行3列
```

♦ **torch.tensor()** 把 Python 列表变张量；**shape** 看形状，NumPy 用户秒懂。

张量的基本运算

Tensor Operations

Notebook 单元格

```
>>> y = x + 10      # 每个元素加 10
>>> z = x * 2       # 每个元素乘 2
>>> m = x @ x.T     # 矩阵乘法 (x 乘 x 的转置)
>>> m.shape
torch.Size([2, 2])
>>> x.sum()         # 全部元素求和
tensor(21)
```

- ◆ 加减乘逐元素、@ 是矩阵乘——这两个“乘法”的区分是考试常客。

与 NumPy 无缝互转

NumPy Interop

Notebook 单元格

```
>>> import numpy as np
>>> arr = x.numpy()    # tensor → numpy
>>> t2 = torch.from_numpy(arr) # numpy → tensor
```

```
# 数据科学生态 (pandas/NumPy) 与
# 深度学习生态 (PyTorch) 由此打通
```

♦ 一行互转：Python 数据科学生态全都是你的后援团。

第一个深度学习程序：Hello Deep Learning

Hello Deep Learning

hello_dl.py · 完整可运行

```
import torch
import torch.nn as nn

model = nn.Sequential(nn.Linear(2, 1))    # 一个最简单的“网络”
X = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y = torch.tensor([[0.], [1.], [1.], [1.]]) # 或运算 OR 的答案
opt = torch.optim.SGD(model.parameters(), lr=0.1)
for epoch in range(200):                # 训练 200 轮
    loss = ((model(X) - y) ** 2).mean()   # 计算误差
    loss.backward(); opt.step(); opt.zero_grad()
print('训练完成! 预测 [1,0] 的结果 =', model(torch.tensor([[1., 0.])))
```

◆ 这 10 行就是一个完整的“训练”——第二章你将逐行读懂它。

这 10 行里藏着整门课的骨架

What Happened Inside

- **nn.Linear(2,1)**: 一个“神经元”——第2章主角
- **X 与 y**: 输入与标准答案（标注），正是“数据+答案”范式
- **loss / backward / step**: 算误差 → 反向传播 → 更新参数，训练三部曲
- 第2、5章将分别讲透其中的原理与技巧；今天先建立“全流程手感”

输出结果示例

- 预测值 ≈ 0.93
- 越接近 1 = 学得越好
- 每次运行略有差异，正常

◆ 今天先跑通“全流程”，原理细节第二章逐行拆解。

互动：让 Hello Deep Learning "活"起来

Lab: Hello DL Live



课堂怎么玩

- 1 点"开始训练", 看 4 个样本被直线分开
- 2 看右侧 loss 曲线一路下降——这就是"学习"
- 3 把 η 拖到 0.8 再训练, 观察 loss 曲线震荡

看什么：刚才那 10 行代码的每一步（前向/损失/反向/更新）都在这页里可视化成了动画。

♦ PPT 里的代码是静态的，这个实验台让同一段代码"活"给你看。

报错锦囊 ①: conda 不是内部或外部命令

Error FAQ 1

原因与诊断

- 用了普通 CMD / PowerShell, 而不是 Anaconda Prompt
- 或安装时未加入 PATH (默认不勾选)
- 诊断: 开始菜单搜 “Anaconda Prompt” 能打开即环境本体正常

1

方案A: 一律改用 Anaconda Prompt (推荐)

2

方案B: 安装时勾选 Add to PATH 后重装

3

方案C: 从开始菜单的 “Anaconda Navigator” 图形界面进入

◆ 记住: 这门课所有命令行操作, 默认都在 Anaconda Prompt 里。

报错锦囊 ②：Python 版本 / 环境混乱

Error FAQ 2

典型症状

- pip 装的包装到了别的环境，dl2026 里 import 失败
- python 命令打开的版本不是 3.10
- 装了半天包，一运行还是 ModuleNotFoundError

1

检查行首是否有 (dl2026)：没有先激活

2

在环境内用 `python -m pip install` 安装

3

还是乱：`conda remove -n dl2026 --all` 删掉重建

♦ 行首没有 (dl2026) 就先激活；环境坏了就重建，3 分钟的事。

报错锦囊 ③：下载超时 / CondaHTTPError

Error FAQ 3

原因

- 默认源服务器在境外，校园网晚高峰不稳定
- conda 与 pip 的代理设置互相干扰

1 配置清华镜像（见第 63 页）后重试

2 pip 慢：加参数 -i 指定清华源

3 换手机热点重试，常常立竿见影

◆ 网络问题换源、换时段、换热点，三板斧解决九成。

报错锦囊 ④：CUDA 相关报错

Error FAQ 4

原因

- 显卡驱动太旧，与新版 CUDA 不匹配
- 装了 CUDA 版但驱动不配套，或装了 CPU 版却调 GPU
- 报错含 “CUDA” “cudnn” “driver” 等关键词

1

本课保底方案：先用 CPU 版，完全不影响今天任务

2

想上 GPU：先更新 NVIDIA 驱动到最新

3

按官网向导重装对应 CUDA 版本，装完跑检测代码

♦ CUDA 问题先降级到 CPU 版保底，课后配好 GPU 再换回。

通用排错方法论：四步走

Debugging Method

1 读报错

从最后一行往上读：错误类型 + 出错文件行号

2 搜索

报错信息原样丢给搜索引擎 / AI 助手，看前 3 条结果

3 最小复现

注释掉可疑代码，二分定位出错的行

4 求助

带上“完整报错截图 + 你做了什么”到课程群提问

♦ 会读报错，是程序员与小白的核心分界线——这门课反复训练它。

随堂练习

5 分钟

Tensor 热身三题 (Notebook 中完成)

- ? 创建形状为 (3,4) 的全零张量，并打印 shape
- ? 创建 [1..9] 的张量，输出其平均值（提示：.float().mean()）
- ? 把上面 Hello Deep Learning 的训练轮数改成 2000，观察预测值变化

提示： torch.zeros(3,4)；第3题体会“训练越久，预测越准”——但也留意它会不会过头。

环境验收清单：今天你应完成

Checklist



Anaconda

conda --version 正常输出版本号



虚拟环境

行首能出现 (dl2026), Python 3.10



PyTorch

import torch 无报错, 版本 2.x



验证程序

张量创建 / 运算 / GPU 检测三个代码段跑通



Hello DL

十行程序运行成功, 预测值接近 1

◆ **五项全勾 = 环境就绪；未完成项课后按报错锦囊补齐。**

实践作业：环境搭建实验记录

Lab Report

- **内容一：** 安装步骤记录（关键截图 2-3 张即可）
- **内容二：** 遇到的报错与解决过程——这条最重要，体现排错思路
- **内容三：** Hello Deep Learning 运行结果截图与一句话解读
- **提交：** Word 或 PDF，下次课前提交到课程群收集表
- **AI 可以辅助排查，但过程记录必须是你的真实经历**

◆ **报错与解决写得越具体，得分越高——真实的踩坑记录最值钱。**

课后任务清单

Homework

1 完成实验记录

按上一页要求，下次课前提交

2 阅读教材第1章

对照三课时内容，标记疑问处下次课提问

3 构思结课项目

从“生活痛点 / 专业结合点”出发，随手记 1-2 个想法

♦ 结课项目从今天就开始发酵：好点子都是平时攒出来的。

结课项目预告：往届风格选题参考

Final Project Seeds

校园类

课堂签到表情识别 · 食堂菜品识别 · 停车位检测

生活类

垃圾分类识别 · 手写笔记数字化 · 简历关键词提取

专业结合类

文献关键词抽取 · 试卷公式识别 · 实验数据预测

创意类

古诗生成 · 校园风景风格迁移 · 简单对话机器人

◆ 现在不必定题——先留心“什么问题让你觉得：这要是能自动就好了”。

课程群 · 答疑 · 办公时间

Communication

- **课程群：**屏幕出示群二维码——所有通知、作业收集、资料分享的唯一渠道
- **提问规范：**报错截图 + 已尝试的操作 + 期望与实际的结果
- **答疑时间：**课后即时答疑；线上问题工作日 24 小时内回复
- **提问也是一门手艺——把问题说清楚，答案就完成了一半**



扫一扫二维码，入群聊



◆ 现在扫码进群，改群名片为“姓名-学号”。

本课引用的经典文献

Classic References

Rosenblatt (1958)

The Perceptron: A Probabilistic Model...

第一个可学习的模型，感知机诞生

Minsky & Papert (1969)

Perceptrons

证明单层感知机局限，第一次寒冬

Rumelhart et al. (1986)

Learning representations by back-propagating errors, Nature

反向传播算法复兴神经网络

LeCun et al. (1998)

Gradient-based learning applied to document recognition

LeNet-5: CNN 的创世纪

Shannon (1948)

A Mathematical Theory of Communication

信息论奠基，熵与交叉熵之源

Krizhevsky et al. (2012)

ImageNet Classification with Deep CNNs

AlexNet 引爆深度学习

Vaswani et al. (2017)

Attention Is All You Need

Transformer 架构，大模型时代

Goodfellow et al. (2016)

Deep Learning (花书)

最权威的深度学习教材

♦ 把公式、模型、算法“钉”回到它们的原始出处——这是研究生阶段最核心的素养。

第一章总结

Day Summary



课时一

定义、三圈关系、应用与边界、判断准则



课时二

三次浪潮、特征学习革命、端到端、预训练微调



课时三

Anaconda + PyTorch 环境就绪，跑通第一个训练程序



带走一句话

深度学习 = 数据 + 答案，让机器自己学出规则



带走一个习惯

读报错、查原因、写记录——工程师的自我修养

♦ 下次课（第2周）：感知机与神经网络，请带好教材与环境。